

Governing how an agent spends

A working Cloudflare Worker, and the product thinking behind it · Nivedita Thapa

The [Agents on Cloudflare post](#) mentioned the open question of what keeps an agent from buying dozens of domains, with the flat \$100 per month cap as the first answer. I got curious about what lives just past that cap, so I built a small layer there to find out. What I kept noticing is that a single number cannot tell the difference between a purchase worth waving through and one worth pausing on.

What it is

A Cloudflare Worker, deployed and running, that decides in real time whether an agent gets to make a given purchase. Every action the agent attempts returns one of three verdicts, each with a logged reason:

- **approve:** when the charge is within budget and within an auto-approve rule. The agent proceeds, no human needed.
- **hold:** when the charge is affordable but consequential. It waits for a one-tap human yes while the agent keeps working on everything else.
- **deny:** when the charge would break the budget.

```
agent goal: build and ship an app on Cloudflare
```

```
[APPROVE] Create R2 bucket for assets (free)
Free service, no spend impact.
$0.00 spent, $100.00 left
```

```
[APPROVE] Register coolapp.dev ($10.44)
Within the $20 auto-approve threshold.
$10.44 spent, $89.56 left
```

```
[HOLD] Workers Paid subscription ($5.00)
Recurring charge, held for human approval.
$10.44 spent, $89.56 left
```

```
[HOLD] Register coolapp.ai ($70.00)
Over the $20 threshold, held for approval.
$10.44 spent, $89.56 left
```

```
[HOLD] Register coolapp.io ($34.00)
Over the $20 threshold, held for approval.
$10.44 spent, $89.56 left
```

The agent working a real purchase queue against the deployed Worker.

Budget and a full audit trail persist in Workers KV. The decision logic lives in its own tested module, so the judgment is inspectable. I stubbed the Stripe identity and payment-token exchange on purpose: the protocol already handles those, and rebuilding them would have been wiring, not insight. The governance was the

part I was curious about, so that is the part I built.

What I noticed

Building it, a few things surfaced that I hadn't expected to matter as much as they did:

- A one-time \$12 domain is done the moment it's bought. A \$12 per month subscription is a standing liability that keeps charging long after the agent has moved on. The charges that tend to surprise people aren't the big one-time ones, they're the small recurring ones nobody's watching. That pushed me to treat recurrence as its own signal, separate from size.
- Registering a domain is hard to undo. Spinning up a bucket is trivial to tear down. How easily an action reverses turned out to be a more useful reason to pause than its price, and it's not something the catalog surfaces today.
- A \$15 charge with \$85 left is routine. The same charge with \$8 left is the one that strands the agent halfway through its task. The decision reads better against the budget that remains than against the sticker price alone.

Product feedback, in priority order

The three things I most wanted while building:

- **A catalog that carries charge semantics:** to make good calls I needed to know whether a charge recurs and how reversible it is, and right now both have to be inferred. If Discovery surfaced that structure, any governance layer on top, yours or a partner's, gets better for free, and agents make smarter calls on their own.
- **A hold, not just a wall:** a cap passes everything beneath it silently and stops everything above it. The state I kept wanting was the one in between, letting the agent keep moving while a single charge waits for a yes. That felt closer to the human-in-the-loop idea the post describes than a hard ceiling does.
- **An audit trail treated as a product:** once agents spend on their own, the question every customer will ask is what did it buy and why was each one allowed. The decision-and-reason record I built for governance turned out to be the thing I'd most want to read back afterward.

Live: agent-spend-governance.niveditathapa.workers.dev/catalog · Code, tests, and a from-scratch deploy guide are in the repo.